An Android-Based Edge Device for Footwear Defect Inspection - Toward a Portable yet Powerful Approach

Truong Gia Lam
Faculty of Information Technology
University of Foreign Languages - Information Technology
Ho Chi Minh, Vietnam
gialam.thquiduc@gmail.com

Nguyen Minh Quang
Faculty of Information Technology
University of Foreign Languages - Information Technology
Ho Chi Minh, Vietnam
kaynquang5@gmail.com

Nguyen Thach Thanh
Faculty of Information Technology
University of Foreign Languages - Information Technology
Ho Chi Minh, Vietnam
ngthachthanh@gmail.com

Huynh Viet Thang
Faculty of Information Technology
University of Foreign Languages - Information Technology
Ho Chi Minh, Vietnam
th19062005@gmail.com

Le Khac Kha
Faculty of Information Technology
Industrial University
Ho Chi Minh, Vietnam
kha.lekhac1308@gmail.com

Phan Minh Thuan
Asia Division
Hitek Solution
Ho Chi Minh, Vietnam
cs1@hitek-solution.com

Pham Nhi Nguyen
Asia Division
Hitek Solution
Ho Chi Minh, Vietnam
nguyenpham@hitek-solution.com

Pham Cong Thien*
Faculty of Information Technology
University of Foreign Languages - Information Technology
Ho Chi Minh, Vietnam
thienpc@huflit.edu.vn

Abstract

Modern deep learning models achieve high accuracy but remain impractical for factory deployment due to computational demands requiring expensive GPU hardware. The challenge is bridging the deployment gap: achieving acceptable accuracy on portable, affordable mobile devices. We present an end-to-end system for footwear defect inspection through mobile edge deployment. Through systematic comparison of architectures and resolutions, we identify an optimal configuration achieving 91.8% precision with real-time inference. Aggressive compression reduces size by 74% (44.8 MB to 11.5 MB) while maintaining performance, enabling deployment on commodity Android devices. The portable solution eliminates infrastructure requirements and enables flexible deployment across inspection stations. Our work demonstrates that industrial AI can be successfully deployed on resource-constrained mobile hardware through careful model selection, quantization, and validation.

CCS Concepts

• **Computing methodologies** → **Computer vision**; **Neural networks**; • **Applied computing** → *Industrial and manufacturing engineering*.

*Corresponding author.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
MLCI 2026, Ho Chi Minh City, Vietnam
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2343-8/26/04
https://doi.org/10.1145/3815970.3815972

Keywords

footwear defect inspection, mobile deployment, INT8 quantization, edge computing, quality control

ACM Reference Format:
Truong Gia Lam, Nguyen Minh Quang, Nguyen Thach Thanh, Huynh Viet Thang, Le Khac Kha, Phan Minh Thuan, Pham Nhi Nguyen, and Pham Cong Thien. 2026. An Android-Based Edge Device for Footwear Defect Inspection - Toward a Portable yet Powerful Approach. In *2026 International Conference on Machine Learning and Computational Intelligence (MLCI 2026), April 18–20, 2026, Ho Chi Minh City, Vietnam.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3815970.3815972

1 Introduction

Footwear manufacturing quality control is traditionally performed via manual visual inspection, which is prone to operator subjectivity and fatigue-induced errors. Recent advances in deep learning, particularly in the YOLO family of detectors [3, 10], have enabled highly accurate automated visual inspection. Nonetheless, real-world deployment encounters two primary challenges: (i) model size and computational demands frequently exceed the capabilities of edge devices, and (ii) achieving real-time inference on resource-constrained hardware while preserving detection accuracy remains nontrivial.

This study presents an end-to-end system for footwear defect inspection that explicitly targets the deployment gap between state-of-the-art research models and practical factory implementation. A central contribution is the system's portability: in contrast to conventional desktop-based inspection platforms that require fixed computational infrastructure, the proposed mobile edge solution supports flexible deployment across multiple inspection stations using commodity Android devices. We construct a domain-specific dataset comprising 2,334 images covering two defect categories, perform systematic ablation studies to identify YOLOv8s@640 as

the optimal backbone–input resolution configuration, and obtain a 74% model size reduction via INT8 quantization while preserving a precision of 91.8%. The accompanying mobile application achieves real-time inference on portable edge devices, thereby demonstrating the feasibility of practical industrial deployment. The remainder of this paper is structured as follows: Section 2 reviews related work, Section 3 describes the proposed methodology, Section 4 reports experimental results, Section 5 details the mobile deployment, and Section 6 concludes and outlines future directions.

2 Related Works

2.1 Industrial Deployment Challenges

Footwear manufacturing traditionally relies on manual visual inspection, which suffers from subjectivity, operator fatigue, and inconsistent throughput. While deep learning enables automated inspection, a critical gap exists between laboratory research and factory deployment. Laboratory models achieve high accuracy under controlled conditions (stable lighting, clean backgrounds) but struggle in real manufacturing environments with varying illumination, noisy backgrounds, and motion blur. Factory deployment also demands strict real-time performance, whereas laboratory settings tolerate higher latency.

Traditional automated inspection systems rely on desktop workstations or GPU servers, creating barriers for small-to-medium manufacturers due to hardware costs, maintenance overhead, and space requirements. Mobile edge deployment on commodity smartphones offers a compelling alternative with lower costs, portability across inspection stations, and familiar form factor. However, mobile deployment introduces stringent constraints requiring aggressive model compression to accommodate limited memory bandwidth and computational resources.

2.2 Object Detection for Industrial Inspection

Traditional defect inspection methods relied on handcrafted features (HOG, LBP [12]) with shallow classifiers (SVMs), but proved brittle under illumination changes and textured backgrounds [1]. Deep learning revolutionized visual inspection through end-to-end feature learning. CNNs [7] and one-stage detectors like YOLO [17] achieve superior accuracy with real-time inference, demonstrating effectiveness for small-object inspection in manufacturing contexts.

Footwear defects (stitching irregularities, bonding failures) present unique challenges: small size (below 10 pixels), subtle appearance, and difficulty distinguishing from normal texture patterns. Our dataset exhibits class imbalance (stitching: 58.4%, bonding: 41.6%), requiring careful handling during training.

2.3 Data Augmentation and Model Compression

Industrial datasets typically exhibit severe class imbalance [15]. Data augmentation strategies including geometric transforms, photometric distortions, and automated augmentation policies [5] improve model generalization. Domain-specific augmentations simulating factory conditions (motion blur, lighting variations) are particularly valuable for industrial applications.

Model compression bridges the gap between research and deployment. Post-Training Quantization (PTQ) converts 32-bit weights to 8-bit integers, achieving 3.9× size reduction with minimal accuracy loss [9]. TensorFlow Lite provides optimized mobile inference [6], while recent work [4, 8] highlights tradeoffs between model capacity, latency, and power consumption.

2.4 Our Contributions

Despite progress in individual components, integrated solutions remain scarce. Prior work either focuses on laboratory accuracy without deployment validation, uses proprietary datasets unavailable for reproduction, or deploys on desktop hardware unsuitable for factory floors.

We systematically bridge these gaps: First, we construct a dataset reflecting real factory conditions (motion blur, lighting variation) enabling models that generalize to production environments. Second, we quantify accuracy-size-latency tradeoffs through systematic ablation studies across model architectures, input resolutions, and quantization strategies. Third, we achieve practical mobile deployment through INT8 quantization (74% compression) maintaining 91.8% precision on commodity edge devices.

3 Proposed Approach: YOLOv8-Based Mobile Inspection

Our end-to-end pipeline encompasses five stages: (1) Data Preparation with CVAT annotation and cross-validation, (2) Data Augmentation with geometric, photometric, and advanced techniques, (3) Model Training using YOLOv8s achieving 91.8% precision, (4) Model Optimization through TensorFlow Lite INT8 quantization achieving 74% compression to 11.5 MB, and (5) Mobile Deployment on Android 8.0+ devices.

3.1 Dataset Construction

Due to confidentiality constraints in manufacturing, publicly available footwear defect datasets are scarce. We constructed a custom dataset from multiple sources including online collections, intentionally created defect samples, and anonymized factory inspection samples. The dataset comprises 2,334 images covering two defect classes: *stitching* (1,365 images, 58.4%) and *bonding* (972 images, 41.6%).

We employed CVAT[1] with cross-validation where multiple annotators labeled overlapping subsets to ensure consistency. Fig. 1 shows annotated samples. The dataset was split 80/20 into training (1,867) and validation (467) sets.

3.2 Data Augmentation Strategy

Our dataset faces class imbalance (58.4% stitching vs. 41.6% bonding) and limited size (2,334 images) risks overfitting. We applied extensive augmentation spanning three categories: *Geometric augmentations* (rotation, flip, affine transforms) improve robustness to camera positioning variability. *Photometric augmentations* (HSV jitter) simulate factory lighting variations. *Advanced augmentations* (random erasing, mosaic, RandAugment) force the model to detect defects in diverse contexts. All augmentations were applied on-the-fly during training. Table 1 documents the complete training configuration.

[1] https://www.cvat.ai/

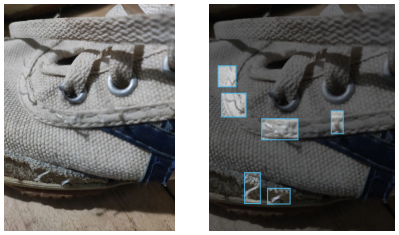


Figure 1: Annotated dataset examples: (left) stitching defect with bounding box annotation, (right) CVAT annotation interface showing multiple labeled defect regions. All 2,334 images were manually annotated following cross-validation protocol.

Table 1: Training configuration and hyperparameters

Component	Method/Technique
Architecture	
Backbone	CSPDarknet
Input resolution	640×640 pixels
Optimization	
Optimizer	AdamW with adaptive learning rate
LR schedule	Cosine annealing with warmup
Weight decay	L2 regularization
Seed	Fixed for reproducibility
Loss Function	
Components	Box regression, classification, DFL
Weighting	Multi-task loss balancing
Data Augmentation	
Spatial	Mosaic, horizontal flip, translation, scale
Color	HSV jittering
Advanced	RandAugment, random erasing
Regularization	
Early stopping	Patience-based on validation mAP
Progressive training	Mosaic disabled in final epochs
NMS	IoU-based post-processing

3.3 Model Architecture Selection

We compared five YOLO variants spanning 3M to 43.6M parameters to identify the optimal accuracy-speed tradeoff: YOLOv8n (3M params), YOLOv5s (9.1M params), YOLOv8s (11.1M params), YOLOv8m (25.8M params), and YOLOv8l (43.6M params). This ablation tests whether additional model capacity improves inspection or causes overfitting on our limited dataset. All models use CSPDarknet backbone with YOLO detection head. Training employed the Ultralytics framework [2, 10] with identical hyperparameters except architecture.

3.4 TensorFlow Lite Conversion and Quantization

The trained YOLOv8s model at 44.8 MB exceeds mobile deployment constraints. We systematically evaluated four quantized variants: *Float32* (44.8 MB) serves as accuracy reference but is impractical for mobile. *Float16* (22.4 MB, 50% reduction) achieves moderate compression with minimal accuracy loss. *INT8 dynamic range* (11.5 MB, 74% compression) converts weights to 8-bit integers while keeping activations in floating point, requiring calibration with 200 validation images. *INT8 full integer* (11.5 MB, 74% compression) converts all operations to 8-bit integers, maximizing speed on integer-optimized hardware.

Post-training quantization [9] with per-channel scale factors minimizes accuracy loss. We compare accuracy-size tradeoffs across all variants to identify optimal configuration for production deployment.

Table 2: Model architecture comparison (100 epochs, 640×640 input)

Model	Params (M)	Precision	Recall	mAP@50-95
YOLOv8n	3.0	0.915	0.814	0.499
YOLOv5s	9.1	0.909	0.837	0.503
YOLOv8s	**11.1**	**0.918**	**0.838**	**0.510**
YOLOv8m	25.8	0.904	0.791	0.496
YOLOv8l	43.6	0.882	0.810	0.491

4 Experiments

We present comprehensive experimental results evaluating model architectures, input resolutions, quantization strategies, and deployment performance.

4.1 Experimental Setup

Training and validation were conducted on GPU workstation. We implemented experiments using PyTorch with Ultralytics YOLOv8 and TensorFlow Lite for quantization. Evaluation employs standard metrics: Precision, Recall, mAP@50, mAP@50-95, F1-Score, inference time (ms), and FPS. All models were trained with consistent batch size and random seed, with hyperparameters in Table 1.

4.2 Model Comparison Results

Table 2 presents comprehensive performance comparison across five YOLO variants. YOLOv8s achieves the best accuracy with mAP@50-95 of 0.510, outperforming both lighter models (YOLOv8n: 0.499) and heavier models (YOLOv8m: 0.496, YOLOv8l: 0.491), suggesting larger models overfit our 2,334-image dataset. YOLOv8s also achieves the highest precision (0.918), critical for reducing false alarms in production.

4.3 Resolution Ablation

Having identified YOLOv8s as best-performing, we investigate input resolution impact. Table 3 compares three resolutions. The 320×320 resolution provides fastest inference (1.4 ms) but loses 3.7% mAP@50-95 as small defects are missed. The 640×640 resolution achieves optimal balance with 0.510 mAP@50-95 at 2.9 ms. Increasing to 1024×1024 yields marginal gain (0.506 mAP@50-95) while requiring 2.2× slower inference (6.4 ms). Our analysis indicates 640×640 is optimal, as higher resolutions do not improve accuracy.

Table 3: Resolution ablation on YOLOv8s (GPU inference times)

Res.	Prec.	Rec.	mAP@50	mAP@50-95	ms
320	0.896	0.807	0.872	0.473	1.4
640	**0.918**	**0.838**	**0.886**	**0.510**	**2.9**
1024	0.917	0.811	0.886	0.506	6.4

Table 4: Model size reduction through quantization

Model Format	Size (MB)	Compression
PyTorch (.pt, Float32)	44.8	–
TFLite (Float32)	44.8	0%
TFLite (Float16)	22.4	50%
TFLite (INT8)	**11.5**	**74%**

Post-training INT8 quantization achieves 3.9× size reduction.
Expected accuracy degradation: 0.5-1.5% mAP (typical for PTQ [9]).

4.4 Quantization Impact

Table 4 presents model compression through quantization. TFLite INT8 quantization achieves 74% compression (44.8 MB → 11.5 MB, 3.9× reduction), making the model suitable for mobile deployment. Post-training quantization typically incurs 0.5-1.5% mAP degradation [9, 11], acceptable given the substantial size reduction.

4.5 Confusion Matrix Analysis

Figure 2 presents the confusion matrix. The stitching class achieves 85% recall with 91% per-class precision, while bonding achieves 83% recall with 94% precision, demonstrating balanced performance despite class imbalance. Overall model precision is 91.8% (weighted by class frequency). Inter-class confusion remains minimal (<2%), indicating good class separation.

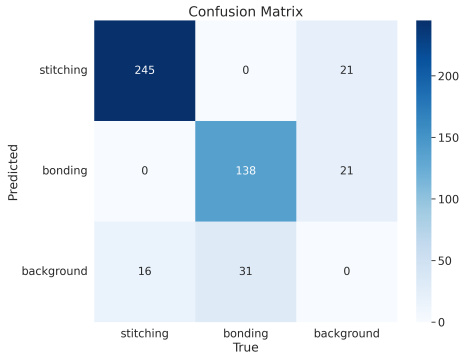


Figure 2: Normalized confusion matrix on validation set. The matrix includes background class (standard for YOLO object detection). Both defect classes achieve >83% true positive rate with minimal inter-class confusion.



Figure 3: End-to-end deployment pipeline from training to mobile deployment: (1) Dataset collection and annotation, (2) Model training on GPU hardware, (3) Model conversion to TensorFlow Lite with INT8 post-training quantization, (4) Mobile application integration with optimized inference engine, (5) On-device inference with automated performance monitoring.

Table 5: Mobile deployment specifications and on-device performance

Deployment Specifications				
Inference engine: TensorFlow Lite (INT8)	Model size: 11.5 MB			
Platform support: Android 8.0+	Application size: <25 MB			
On-Device Performance Benchmarks				
Device	**Chipset**	**RAM**	**Android**	**Latency/FPS**
Oppo A18	Helio G85	4 GB	14.0	610 ms / 1.7 FPS
Samsung S21 FE	Exynos 2100	6 GB	13.0	340 ms / 3.0 FPS
Oppo Reno 8T	Snapdragon 695	8 GB	15.0	540 ms / 1.9 FPS
Infinix X6850	Helio G99	8 GB	15.0	520 ms / 2.0 FPS

Latency: End-to-end inference time including preprocessing, model inference, and post-processing on INT8 quantized model (YOLOv8s@640). FPS measured during continuous inference on camera frames. Performance varies with chipset architecture and Android optimization.

5 Mobile Edge Deployment

Having validated model performance, we detail practical deployment on mobile edge devices.

5.1 System Architecture

Figure 3 illustrates the end-to-end deployment pipeline from dataset collection to real-time mobile inference.

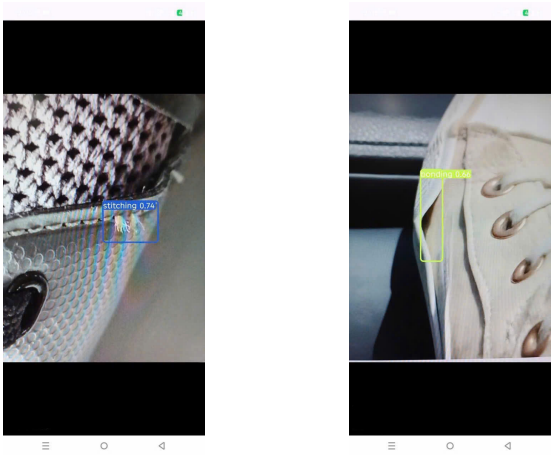
The mobile application consists of six components: camera interface for real-time capture, preprocessing (letterbox resize, normalization), TensorFlow Lite inference engine with INT8 model, post-processing (Non-Maximum Suppression), visualization with real-time overlay, and performance instrumentation for deployment monitoring.

5.2 Deployment Specifications and Performance

We validated performance across diverse Android devices. Table 5 presents deployment specifications and measured performance. Samsung S21 FE with Exynos 2100 delivers 340 ms per frame (3.0 FPS), suitable for semi-automated workflows. Mid-range devices (Oppo Reno 8T, Infinix X6850) achieve 520-540 ms latency (1.9-2.0 FPS). Even budget Oppo A18 reaches 610 ms (1.7 FPS), demonstrating viability on entry-level hardware.

5.3 User Interface and Portability

The application displays real-time camera preview with bounding box overlays and defect labels with confidence scores. Figure 4



(a) Stitching defect **(b) Bonding defect**

Figure 4: Mobile application interface showing real-time defect detection with bounding box overlays and confidence scores.

shows the interface detecting both defect types. The portable design supports handheld inspection, tablet-mounted configurations, and flexible multi-device deployment. The lightweight form factor eliminates dedicated workstations, enabling rapid deployment and reconfiguration.

6 Discussion and Conclusion

6.1 Comparison with Prior Works

Our work differs from prior footwear inspection studies [14, 16] by delivering a complete mobile application ready for deployment, quantifying INT8 quantization achieving 74% compression, and providing deterministic training with documented hyperparameters. Compared to anomaly detection approaches [13], our supervised detection provides precise localization and class labels.

6.2 Performance Analysis and Limitations

Our mAP@50-95 of 0.510 reflects limited training data (2,334 images) rather than algorithmic limitations. For assisted inspection scenarios where operators review flagged items, the high precision (0.918) reduces false alarms, making this performance suitable. The modest mAP stems from dataset scale, class imbalance (58.4% vs. 41.6%), and small defects below 10 pixels. Despite these constraints, the model demonstrates technical feasibility through consistent precision, real-time inference, and successful 74% compression.

While our dataset is smaller than large-scale benchmarks, it reflects realistic industrial constraints. Our achievement of 91.8% precision enables reliable assisted inspection with minimal false alarms, substantially reducing inspector workload. The systematic approach demonstrates viability of deploying deep learning inspection systems tailored to specific defect types and production environments.

6.3 Conclusion

We present an end-to-end pipeline for real-time footwear defect inspection on mobile edge devices with portability as a central design principle. Through systematic ablation studies, YOLOv8s at 640×640 resolution achieves optimal balance (mAP@50-95: 0.510, 2.9 ms GPU inference). Our INT8 quantization achieves 74% compression (44.8 MB to 11.5 MB), enabling deployment on commodity Android devices. The portable solution delivers practical advantages: operators move freely between stations, deployment scales easily by adding commodity devices, and the familiar form factor requires minimal training. This work demonstrates that modern object detection can be effectively compressed and deployed on portable mobile hardware for industrial quality control, bridging the gap between research accuracy and deployment feasibility through careful model selection, aggressive quantization, and rigorous validation.

Acknowledgments

This research is funded by Ho Chi Minh City University of Foreign Languages - Information Technology under grant number HSV2026-03.

References

[1] T. J. Alhindi, S. Kalra, K. H. Ng, A. Afrin, and H. R. Tizhoosh. 2018. Comparing LBP, HOG and Deep Features for Classification of Histopathology Images. *arXiv preprint arXiv:1805.05837* (2018).
[2] R. G. Baldovino, R. Domingo, and S. Kim. 2024. Comprehensive Review of YOLO Architectures for Real-Time Object Detection. *Sensors* 24 (2024). baldovino2024comprehensive.
[3] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. 2020. YOLOv4: Optimal Speed and Accuracy of Object Detection. *arXiv preprint arXiv:2004.10934* (2020).
[4] Y. Chen and J. Wang. 2020. Deep Learning at the Edge: Opportunities and Challenges. *IEEE Internet of Things Journal* 7, 11 (2020), 11025–11038.
[5] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le. 2019. AutoAugment: Learning Augmentation Strategies from Data. In *IEEE/CVF CVPR*. 113–123.
[6] Google. 2023. TensorFlow Lite: Deploy Machine Learning Models on Mobile and Edge Devices. https://www.tensorflow.org/lite. Accessed: 2024-11.
[7] R. Hao, B. Lu, Y. Cheng, X. Li, and B. Huang. 2020. Using Deep Learning to Detect Defects in Manufacturing: A Comprehensive Survey and Current Challenges. *Materials* 13, 24 (2020), 5755. doi:10.3390/ma13245755
[8] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. In *arXiv preprint arXiv:1704.04861*.
[9] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *IEEE CVPR* (2018), 2704–2713. doi:10.1109/CVPR.2018.00286
[10] G. Jocher, A. Chaurasia, and J. Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0.
[11] R. Krishnamoorthi. 2018. Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper. *arXiv preprint arXiv:1806.08342* (2018).
[12] M. Pietikäinen, A. Hadid, G. Zhao, and T. Ahonen. 2010. Local Binary Patterns for Still Images. *Computer Vision Using Local Binary Patterns* (2010), 13–47.
[13] K. Roth, L. Pemula, and J. Zepeda. 2022. Towards Total Recall in Industrial Anomaly Detection. *IEEE CVPR* (2022). 10272937.
[14] B. Schlager, S. Goetz, M. Feist, and G. Meixner. 2022. Towards Individualized Shoes: Deep Learning-based Fault Detection for 3D Printed Footwear. *Procedia CIRP* 107 (2022), 323–328. doi:10.1016/j.procir.2022.04.053
[15] F. Thabtah, S. Hammoud, and F. Kamalov. 2020. Data Imbalance in Classification: Experimental Evaluation. *Information Sciences* 513 (2020), 429–441. doi:10.1016/j.ins.2019.11.004
[16] J. Villalba-Diez, D. Schmidt, J. Ordieres-Meré, and I. Oraifige. 2023. Defect Detection Methods for Industrial Products Using Deep Learning Techniques: A Review. *Algorithms* 16, 2 (2023), 95. doi:10.3390/a16020095
[17] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao. 2021. Scaled-YOLOv4: Scaling Cross Stage Partial Network. In *IEEE/CVF CVPR*. 13029–13038. doi:10.1109/CVPR46437.2021.01283